12 — REACT ROUTER

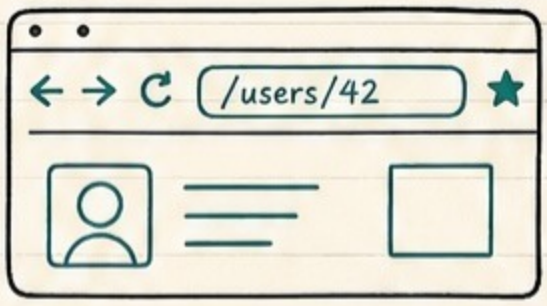
→ Routing maps URLs to pages/components in a single-page app.

DEFINITION:

Browser URL → Router matches route → renders element; navigation without full reload

WHY:

- multi-page UX inside SPA
- shareable links
- back/forward works



CORE PIECES:

BrowserRouter | Routes/Route | Link/NavLink | useNavigate | useParams | Outlet (nested)

FLOW:

URL /users/42 → match Route path → UserPage params.id = 42

CODE SKETCH:

```
<Routes>
  <Route path="/" element={<Home/>} />
  <Route path="/users/:id" element={<User/>} />
  <Route path="*" element={<NotFound/>} />
</Routes>
```

:id becomes params.id inside User component

NESTED ROUTES:

Layout with <Outlet/> for shared nav

```
<Route path="/" element={<AppLayout/>}>
  <Route index element={<Home/>} />
  <Route path="dashboard" element={<Dashboard/>} />
  <Route path="users" element={<UsersLayout/>}>
    <Route index element={<UsersList/>} />
    <Route path=":id" element={<User/>} />
  </Route>
</Route>
```

AppLayout.jsx (example)

```
return (
  <div>
    <Navbar />
    <hr />
    <Outlet />  {/* child routes render here */}
    <Footer />
  </div>
);
```

INTERVIEW NOTES:

- Client vs Server Routing
  Client: handled in browser (React Router).
  Server: each navigation hits server, returns new HTML.
- Protected Routes Pattern
  Wrap routes in a guard that checks auth and either renders element or redirects.

RequireAuth

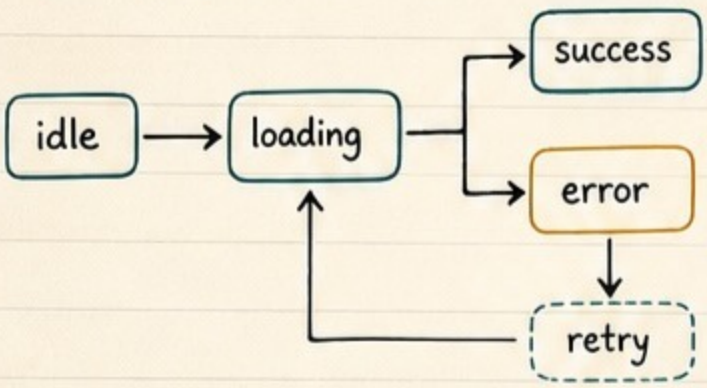
COMMON MISTAKES:

- ✕ using <a href> causing full refresh
- ✕ wrong nesting (Outlet missing or misplaced)
- ✕ not handling 404 ("*") route
- ✕ hardcoding URLs instead of using Link
- ✕ forgetting trailing slash / vs missing params in useParams()

13 – DATA FETCHING

★ Loading remote data needs clear states: loading, success, error — and cleanup.

• STATES (state machine)



• BASIC PATTERN

```
useEffect(() => {
  const controller = new AbortController();
  const { signal } = controller;

  const load = async () => {
    try {
      setState(s => ({ ...s, loading: true, error: null }));
      const res = await fetch('/api/products', { signal });
      if (!res.ok) throw new Error('Network error');
      const data = await res.json();
      setState({ data, loading: false, error: null });
    } catch (err) {
      if (err.name === 'AbortError') return;
      setState({ data: [], loading: false, error: err.message });
    }
  };
  load();
  return () => controller.abort();   // cleanup on unmount
}, []);
```

- works the same with axios:

```
axios.get(url, {
  signal
})
```

• BETTER OPTIONS

① Custom useFetch hook

- Encapsulate state machine
- Reuse across components
- Add abort, retry, transform

Good for learning + small apps.

② React Query / TanStack Query

- Caching
- Deduping
- Retry + backoff
- Stale-while-revalidate
- Background refetch

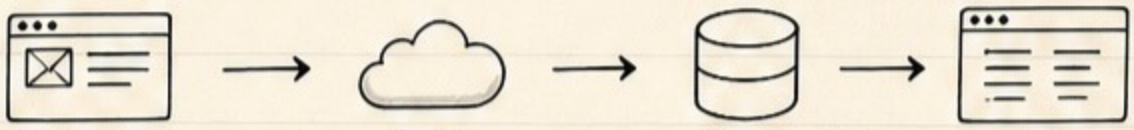
Most production apps use this.

③ Server Components / Loaders (framework dependent)

- Fetch on server
- Stream or hydrate
- Less client JS, better SEO

Next.js App Router, Remix, React Router loaders, etc.

• REQUEST / RESPONSE FLOW



on unmount



abort request (cleanup)

• PRACTICAL EXAMPLE — Product List

```
function ProductList() {
  const [state, setState] = useState({
    data: [], loading: false, error: null
});
useEffect(() => {
  const controller = new AbortController();
  const load = async () => {
    setState(s => ({ ...s, loading: true, error: null }));
    try {
      const res = await fetch('/api/products', { signal: controller.signal });
      if (!res.ok) throw new Error('Failed to load');
      const data = await res.json();
      setState({ data, loading: false, error: null });
    } catch (err) {
      if (err.name !== 'AbortError') {
        setState({ data: [], loading: false, error: err.message });
      }}
  };
  load();
  return () => controller.abort();
}, []);

if (state.loading) return <Spinner />;
if (state.error) return (
  <ErrorBanner
    message={state.error}
    onRetry={() => setState(s => ({...s, data: [], error: null, loading: false }))
      setTimeout(() => {}, 0) /* trigger effect by remount or key change */
  }
);
return (
  <ul className="grid">
    {state.data.map(p => (
      <li key={p.id} className="card">
        <img src={p.image} alt={p.name} />
        <div>{p.name}</div>
        <div className="price">${p.price}</div>
      </li>))}
```

• UI PREVIEW

Loading state



Loading products...

Error state



Failed to load products. Retry

Success state



Sneaker $59.99



Backpack $39.99



Headphones $29.99

... more products

ADVANTAGES of QUERY LIBS

- Caching (memory)
- Deduping requests
- Background refresh
- Retry + backoff
- Window focus refetch
- Pagination / infinite scroll
- Devtools (inspector)

★ Less boilerplate!

LIMITS

- More setup / learning
- Library size
- Overkill for tiny apps

Understand basics first.

19 — ROADMAP & MASTER RECAP

LEARNING PATH (WEEKS)

- **W1** — JSX & components & props
- **W2** — useState events, forms, lists
- **W3** — useEffect & hooks rules
- **W4** — Context, refs, memo
- **W5** — Router & fetching
- **W6** — Patterns & state choices
- **W7** — Mini app + interview polish

MUST-KNOW CHECKS

- ☑ UI = f(state)
- ☑ props vs state
- ☑ keys for lists
- ☑ effects need cleanup
- ☑ controlled inputs
- ☑ context sparingly
- ☑ memo with measure

COMMON MISTAKES ✗

- ✗ mutate state
- ✗ index keys always
- ✗ fetch in render
- ✗ giant context
- ✗ premature Redux
- ✗ ignore cleanup

UI = F(STATE). KEEP COMPONENTS SMALL. THINK IN HOOKS.

04 – STATE with useState

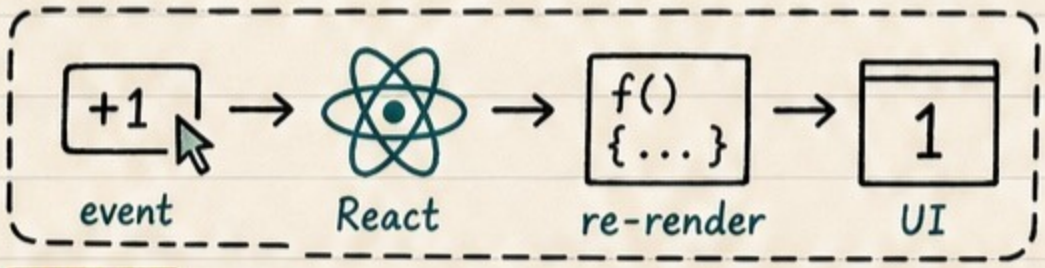
State is data that changes over time and belongs to a component.

DEFINITION `useState(initial) → [value, setValue]`

WHY : interactive UI (inputs, toggles, carts)

HOW IT WORKS

1. Event calls setState
2. React schedules re-render
3. Component function runs again with new state
4. UI matches latest state



RULES

- state updates async / batched
- use functional updates when next depends on prev
- don't mutate arrays / objects – replace them
- each render has its own state snapshot

CODE

```
const [count, setCount] = useState(0);
<button onClick={() => setCount(c => c + 1)}>{count}</button>
```

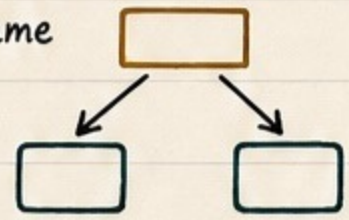
WHEN LOCAL STATE

- form fields
- open/close UI
- temporary selections



WHEN LIFT STATE

two children need same data → move up



INTERVIEW

Q: Why immutability?

A: so React can detect changes reliably.

MISTAKE

- mutating state then wondering why no re-render
- storing derived data unnecessarily

02 — JSX & ELEMENTS

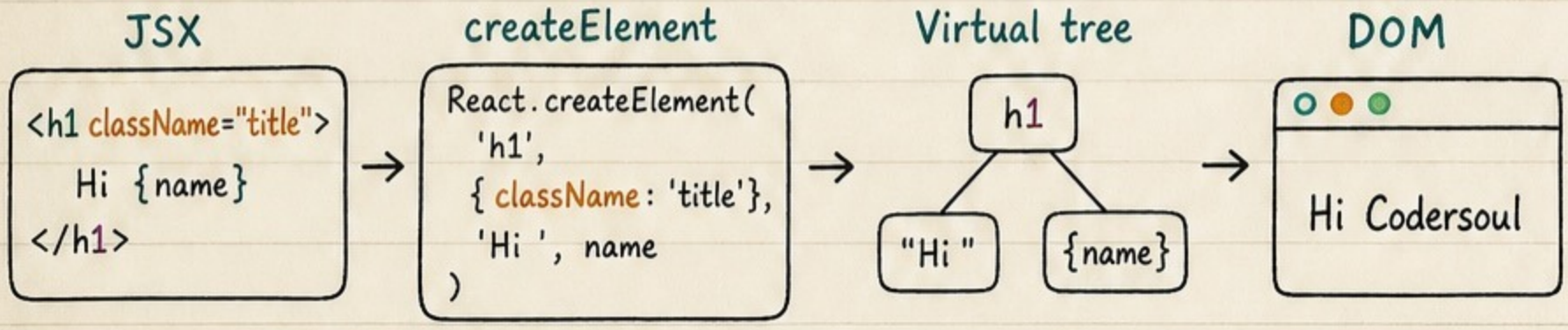
→ JSX is syntax sugar that looks like HTML in JavaScript — it becomes React elements.

- **DEFINITION:** JSX → React.createElement(...) → element objects describing UI
- **WHY IMPORTANT:** readable UI structure next to logic
- **HOW IT WORKS:**
  ① Write JSX
  ↓
  ② Babel/compiler transforms
  ↓
  ③ React creates element tree
  ↓
  ④ Renderer paints DOM/native

RULES

- One parent (or Fragment <>)
- camelCase props: `className`, `onClick`
- JS expressions in { }
- self-close void tags `<img />`

```
1  function Hello({ name }) {
2      return <h1 className="title">Hi {name}</h1>;
3  }
```



★ INTERVIEW

JSX is NOT HTML; it's JS.

It compiles to

⊗ MISTAKE

- using `class=` instead of `className`
- forgetting `keys` later in lists
- if statements inside

★ TAKEAWAY

JSX describes UI. Expressions in braces bring it

06 — CONDITIONAL UI & LISTS

Show UI based on conditions.
Render lists from arrays with stable keys.

• CONDITIONAL PATTERNS

1 && for simple show/hide

```
{isOpen && <Modal/>}
```

2 ternary

a ? <A/> : <B/>

```
{isAuth ? <App/> : <Login/>}
```

3 early return

if (!data) return <Loading/>

```
if (!data) return <Loading/>
```

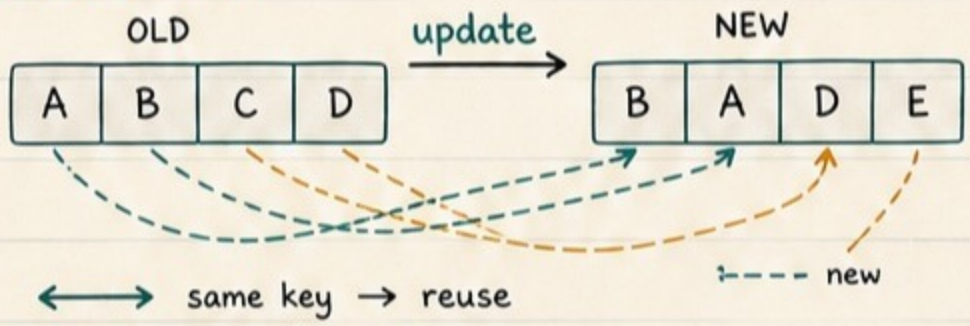
4 switch / lookup map for many states

```
// see example below
```

• LISTS

```
items.map(item => <Row key={item.id} ... />)
```

• KEYS WHY



⟷ same key → reuse
⟵⟶ removed
•⟶ new

keys help React match old/new list items → correct reuse / update / delete

• KEY RULES

- ✓ Use stable, unique ids from your data.
- ✗ NEVER use random() as key.
- ✓ Index as key ONLY if list is static (never reorders / filters / inserts).

• INTERVIEW

Why not use index as key when sorting / filtering? ?

↳ Because items change position → React reuses wrong component instances → state/inputs can stick to the wrong item.

• MISTAKES

- ✗ Missing key → React warning (ignored = future bugs).
- ✗ Huge nested ternaries → hard to read & maintain

• EXAMPLE — TODO LIST FILTER

```
const [filter, setFilter] = useState('all')
const todos = [
  { id: 1, text: 'Learn React', done: false },
  { id: 2, text: 'Build App', done: true },
  { id: 3, text: 'Deploy', done: false },
]
const visible = todos.filter(t =>
   filter === 'all' ? true :
   filter === 'active' ? !t.done :  t.done
)
return (
  <div>
    <div className="filters">
      {['all', 'active', 'done'].map(f => (
        <button key={f}
          onClick={() => setFilter(f)}>{f}</button>
      ))}
    </div>
    <ul>
      {visible.map(todo => (
        <li key={todo.id} className={todo.done ? 'done' : ''}>
          <input type="checkbox" checked={todo.done} readOnly />
          {todo.text}
        </li>
      ))}
    </ul>
  </div>
)
```

• COMPLEXITY

15 – Component Patterns

★ Patterns keep components flexible without messy prop explosions.

PATTERNS

#	PATTERN	HOW (TL;DR)	WHY	EXAMPLE

1	Composition with children 	Parent passes JSX as children. Child decides where to render it.	Flexible layouts, reusable wrappers, less prop drilling.	`<Card>`
				`  <Title />`
				`  <Body />`
				`</Card>`

2	Container vs Presentational (smart / dumb)	Container handles state/ effects. Presentational just renders props. Still useful mentally.	Separation of concerns, easier testing & reuse.	`<UserContainer />` (fetches user) `<UserView user={user} />` (pure UI)

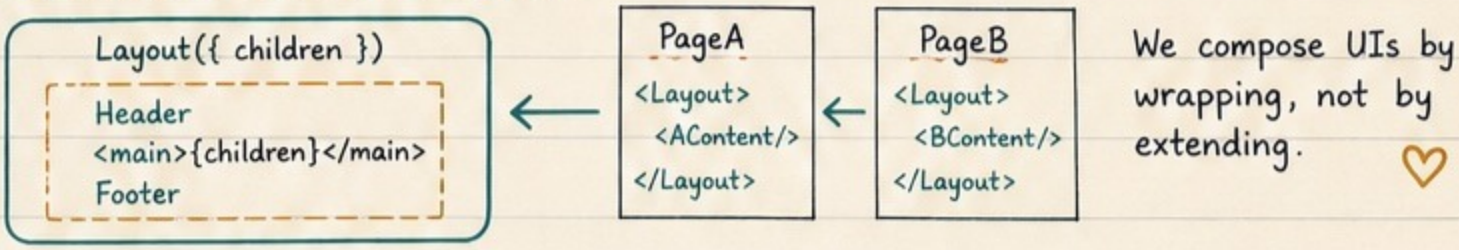
3	Controlled Components Pattern `<input value=... />`	Form state lives in parent. Value + onChange come from props.	Predictable data flow, validation, single source of truth.	`<Input`
				`  value={name}`
				`  onChange={setName}`
				`/>`

4	Compound Components (Tabs pattern)	Parent provides context. Subcomponents communicate via context (or props).	Clear API surface, great DX, encourages correct usage.	`<Tabs defaultValue="1">`
				`<Tabs.List>...</Tabs.List>`
				`<Tabs.Panel value="1">...</Tabs.Panel>`
				`</Tabs>`

5	Render Props / Headless ({ data }) => ...	Component exposes logic via a function as child (render prop) or hook (headless component).	Maximum flexibility, reusable logic without opinionated UI.	`<DataFetcher>`
				`{({ data, loading }) =>`
				`loading ? <Spinner/> : <UI data={data}/>`
				`}`
				`</DataFetcher>`

6	Custom Hooks Extract Logic (useSomething()	Move stateful logic, effects, subscriptions into a hook.	Share logic across components, clean components, testable.	`function useForm(initial) {`
				`// logic`
				`return { values, handleChange, reset }`
				`}`

★ COMPOSITION OVER INHERITANCE



We compose UIs by wrapping, not by extending. ♡

PRACTICAL EXAMPLES

- Modal with children content

```
<Modal open={open} onClose={close}>
  <h2>Confirm</h2>
  <p>Are you sure?</p>
  <Button onClick={close}>Close</Button>
</Modal>
```



- Button variants via props

```
<Button variant="primary" size="md"> Save</Button>
```

Primary | Secondary | Ghost | Danger

INTERVIEW FAVORITES

- What is composition vs inheritance in React?
  → Composition lets us build UIs by combining components. No inheritance. We pass children and props.
- What is the children prop?
  → It's a special prop that lets components receive JSX inside them: <Comp> ... </Comp>

⚠ COMMON MISTAKES

- God components: doing too much (data, UI, logic, styles).
- Prop drilling 8 levels deep before using context/composition.
- Premature abstraction: building a pattern before the need.

ADVANTAGES

⚠ LIMITS

03 – Components & Props

> ★ Components are reusable UI pieces. Props are inputs from parent to child.

DEFINITION

A component is a JavaScript function that returns React elements; props are read-only data passed to the component.

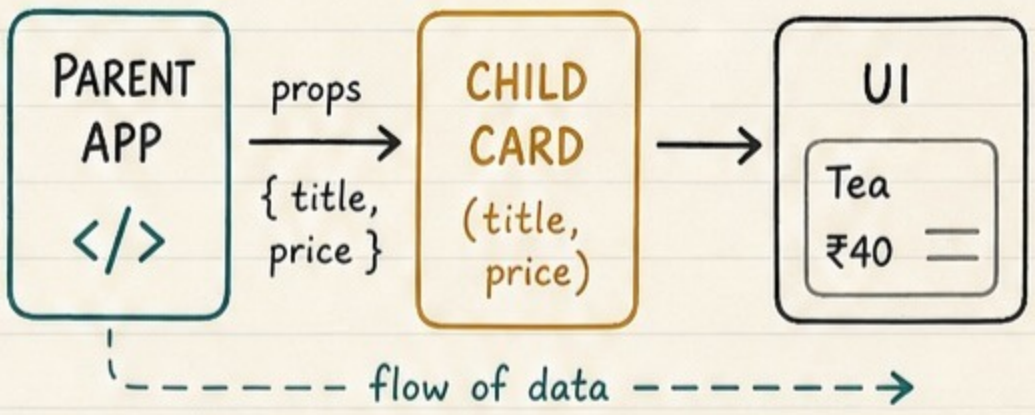
WHY

Split UI into small, named, testable units.

HOW IT WORKS

1. Parent passes props
2. Child renders using props
3. Data flows down

DATA FLOW (ONE WAY ↓)




```
function Card({ title, price }) {
  return (
    <article>
      <h3>{title}</h3>
      <p>{price}</p>
    </article>
  );
}
// <Card title="Tea" price="₹40" />
```

PROPS RULES

- Props are read-only.
- Don't mutate props.
- Destructure for clarity.
- Use children prop for nesting and composition.

COMPOSITION > PROPS SOUP

Prefer composition over passing too many props (a.k.a. props soup). Build small components and combine them. It's more flexible and maintainable.

INTERVIEW CORNER

Q: Props vs State?

- Props = inputs from outside (parent)
- State = owned inside the component and can change over time.

COMMON MISTAKES

- Mutating props (breaks one-way flow).
- Panic about prop drilling too early. Solve it when/if it actually hurts.

ADVANTAGES

- ✓ Reusability
- ✓ Separation of concerns

LIMITS

- Prop drilling in deep trees can be noisy.
- Not ideal for frequently changing data (use state/context)

18 — CHEAT SHEET + INTERVIEW

Copy. Build. Explain.

① HOOKS QUICK

- useState
- useEffect
- useRef
- useMemo
- useCallback
- useContext
- useReducer

Know what & when to use.

② JSX RULES

- One parent element
  `<div> ... </div>`
- className (not class)
- Use { } to embed JS in JSX
  `<h1>{title}</h1>`
- keys on lists
  `key={item.id}`

③ STATE RULES

- State is immutable. Never mutate existing state.
  `arr.push(x)` ✗
  `setArr([...arr, x])` ✓
- Use functional updates when next state depends on previous.
  `setCount (c => c + 1)`

④ EFFECT RULES

- useEffect syncs with external systems (DOM, APIs, Subscriptions).
- Always cleanup to avoid memory leaks.
  `return () => { cleanup }`
- Dependencies must be honest. No cheats.
  `useEffect(() => { ... }, [dep])`

⑤ ROUTER MINI

- Routes → Wrap all routes
  `<Routes>...</Routes>`
- Route → Define a path
  `<Route path"=/" element={<Home />} />`
- Link → Navigate
  `<Link to="/about"> About</Link>`
- useParams → Read params
  `const { id } = useParams()`
- useNavigate → Programmatic nav
  `const navigate = useNavigate()`
  `navigate ("/dashboard')`

⑥ FETCH STATES

LOADING
→ Show spinner / skeleton while fetching.

SUCCESS
→ Show data to user.

ERROR
→ Show error message + retry option.

Tip:
Handle all 3 states in UI.

⑦ TOP INTERVIEW Qs

① What is Virtual DOM?
A lightweight JS representation of the real DOM. React updates the VDOM first, then patches only what changed in real DOM.

② What are keys & why important?
Keys help React identify items in a list. They enable efficient re-renders & preserve component state.

③ What is a controlled input?
An input whose value is driven by React state.
`value={value} onChange={handleChange}`

④ Props vs State?
Props are read-only inputs from parent.
State is managed within the component, can change.

⑤ Why follow Rules of Hooks?

⑧ COMMON MISTAKES

- ☐ Mutating state directly.
- ☐ Missing or wrong dependencies in useEffect.
- ☐ Using index as key in lists.
- ☐ Conditional Hook calls.
- ☐ Calling Hooks inside loops / nested functions.
- ☐ Not cleaning up effects (subscriptions, timers).
- ☐ Setting state after unmount (memory leaks).
- ☐ Overusing state (derive instead of storing).
- ☐ Not handling loading / error states.

10 – useRef & DOM ACCESS

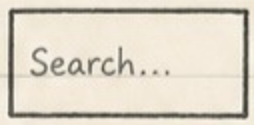
useRef holds a **mutable** value that persists across renders without causing re-renders.

DEFINITION:	const ref = useRef(initial); ref.current is **mutable**

TWO USES:

1 DOM nodes

ref={inputRef}



then

inputRef.current.focus()

2 Store timers / previous values without re-render



setTimeout, intervals, prev values, IDs, etc.

- **WHY IMPORTANT:** escape hatch for imperative APIs; **not** for regular UI state.
- **HOW IT WORKS:** assign ref → React sets .current after mount → read/write .current in events/effects.
- **CODE:** focus search box on mount

```
import { useRef, useEffect } from 'react';
export default function Search() {
  const inputRef = useRef(null);
  useEffect(() => {
    inputRef.current.focus();
  }, []);  // run once after mount
  return (
    <input ref={inputRef} placeholder="Search..." />
  );
}
```

- **COMPARE:**

HOOK	WHAT CHANGES DO?	EFFECT
useState	changes re-render UI	UI updates
useRef	changes silent, no re-render	UI unchanged

- **INTERVIEW:**

When would you use a ref instead of state?

→ DOM access, timers, previous values, caches, imperative handles

- **COMMON MISTAKE:**
  - using ref for values that should update the UI.
  - reading ref.current during render for display logic carelessly.

11 — Memoization & Re-Renders

React re-renders often. Memoization **skips wasted work** when inputs are unchanged.

What Triggers Re-Render:

- setState
- parent re-render
- context change

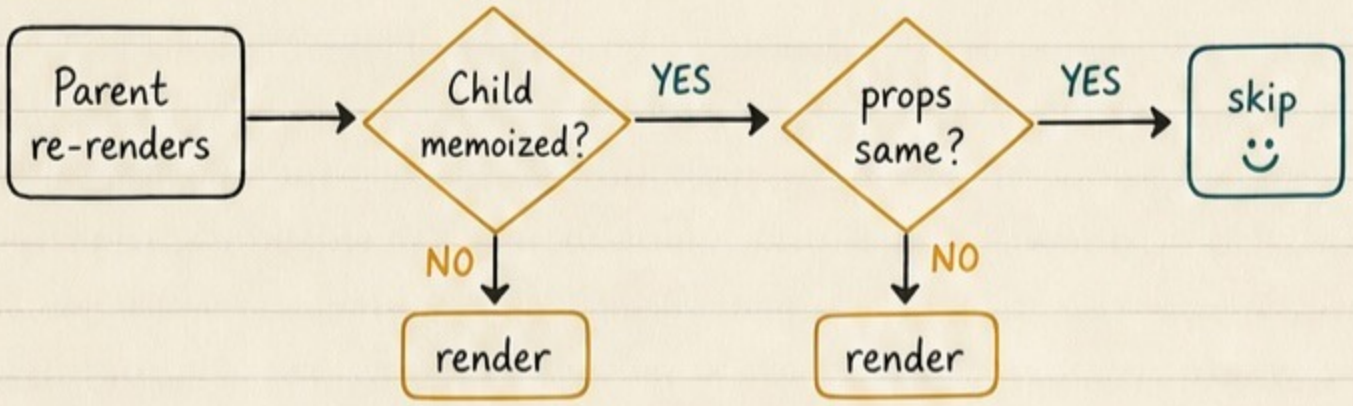
Tools (your friends 💪)

1 `React.memo(Component)`
skip re-render if props shallow-equal

2 `useMemo(() => calc, deps)`
cache expensive calculation

3 `useCallback(fn, deps)`
stable function identity for child memo

How it works (mental model):



✓ When to use

- measured expensive lists/charts (filtering, sorting, aggregations)
- stable callbacks to memo children that rely on referential equality

✗ When not to use

- premature optimization on cheap components
- if it adds complexity without real user-perceived benefit

Code Examples

useMemo – cache expensive filtered list

```
function List({ items, query }) {
  const filtered = useMemo(() => {
    const q = query.toLowerCase();
    return items
      .filter(i => i.name.toLowerCase().includes(q))
      .sort((a, b) => a.name.localeCompare(b.name));
  }, [items, query]);

  return <ListView items={filtered} />;
}
// Re-compute only when items or query changes
```

useCallback – stable onClick for memo child

```
function Parent({ id }) {
  const handleClick = useCallback((itemId) => {
    console.log('clicked', itemId, id);
  }, [id]);

  return (
    <MemoItem id={id} onClick={handleClick} />
  );
}

// handleClick identity stable unless `id` changes
```

Interview Focus 🔍

- Understand referential equality (=== for objects/functions)
- Why inline objects/functions break memoization
- How React.memo does a shallow

Common Mistakes ⚠

- Wrapping everything in useMemo "just in case"
- Missing deps → stale values or bugs
- Thinking memo fixes all perf issues (it doesn't!)

Complexity Note 📊

Expensive operations like filter/sort/map/reduce are typically $O(n \log n)$ or $O(n)$. Good candidates for memoization when n is large and dependencies

Perf Tip ★

Measure with React DevTools Profiler.

Look for components that re-render often with same props or expensive calculations

01 — WHAT & WHY REACT

React is a library for building interactive UIs with reusable components.

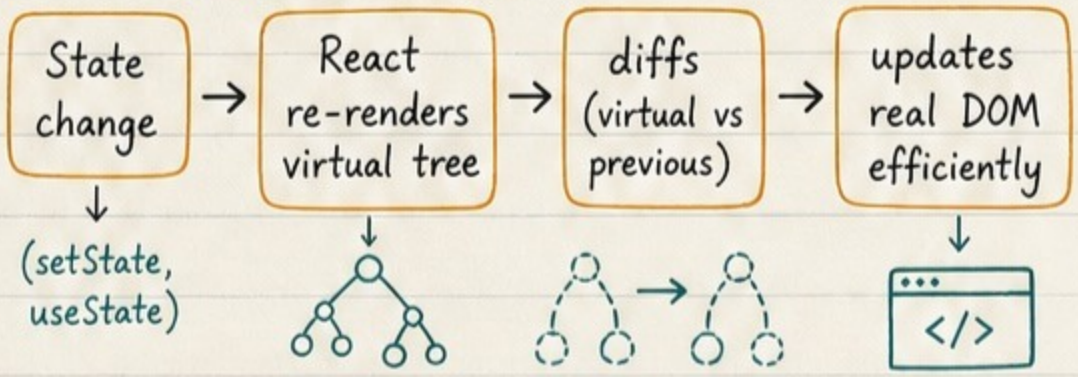
DEFINITION →

- library for composing UIs from components.
- UI updates when state changes.

WHY IT MATTERS

- ☆ Component reuse → faster dev.
- ☆ Predictable UI = f(state)
- ☆ Huge ecosystem & community
- ☆ Interview staple for FE roles

HOW IT WORKS



CORE IDEAS

Components

Break UI into small, reusable, isolated pieces.

Declarative UI

You describe what UI should look like for a given state.

Unidirectional data

Data flows down (props). Events flow up (callbacks).

Hooks

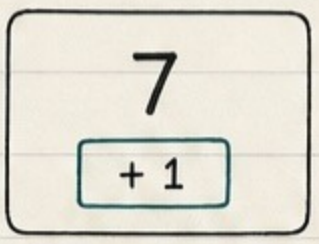
Use state + other React features in function components.

REAL EXAMPLE

Counter button updates number on click — state drives UI. →

```
const [count, setCount] = useState(0);
return (
  <div>
    <h1>{count}</h1>
    <button onClick={() => setCount(c => c+1)}>
      + 1
    </button>
  </div>
);
```

→



Click → count increases, UI updates.

ADVANTAGES

- ✓ Reusable components = faster development
- ✓ Predictable mental model (state → UI)
- ✓ Easy to test components
- ✓ Rich tooling (DevTools, ESLint, Vite, etc.)
- ✓ Large community & countless resources

LIMITATIONS

- Not a full framework. (Routing, data fetching, etc. you choose)
- Learning curve for hooks & render model (deps, closure, batching)

INTERVIEW

Q. What problem does React solve?

COMMON MISTAKE

Treating React like jQuery —

08 — HOOKS RULES & CUSTOM HOOKS

Hooks let function components use state/effects.
Rules keep React's call order reliable.

RULES OF HOOKS

1. Only call hooks at top level (no if/loops)
2. Only call from React functions / custom hooks

WHY: React relies on call order between renders.

CUSTOM HOOKS function starting with use that composes built-in hooks — share logic not UI.

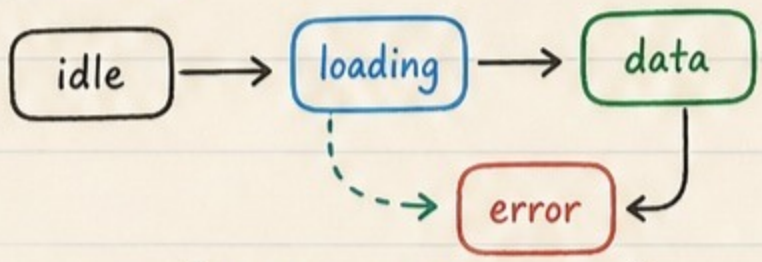
EXAMPLE useLocalStorage(key, initial)



- reads / writes localStorage
- returns [value, setValue]

```
const [value, setValue] =
  useLocalStorage('theme', 'light')
```

EXAMPLE useFetch(url)



returns { loading, data, error }

WHEN EXTRACT : same stateful logic in 2+ components.

INTERVIEW : Can hooks be conditional? → No — breaks order.

MISTAKE :
- naming custom hook without use
- sharing state accidentally via module vars

ADVANTAGES :
- reusable logic
- testable

LIMITS : don't over-abstract early.

TAKEAWAY

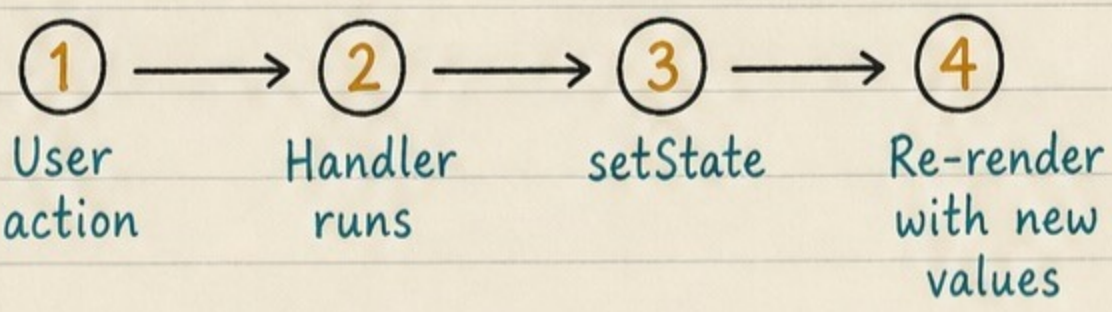
05 — EVENTS & FORMS (BASICS)

React events are named in camelCase and pass a SyntheticEvent wrapper.

DEFINITION : onClick, onChange, onSubmit handlers update state → UI updates

WHY : controlled interactions without direct DOM hacks

HOW :



CONTROLLED INPUT

`<input />` → `value={text}` `onChange={e => setText(e.target.value)}` → state is single source of truth

```
<form onSubmit={handleSubmit}>
   <input value={email} onChange={e => setEmail(e.target.value)} />
   <button type="submit">Save</button>
</form>
// preventDefault() on submit
```

☆ **PRACTICAL** : login email field stays in sync with state

👍 **ADVANTAGES** : predictable testing & validation

⚠ **LIMITS** : more boilerplate than uncontrolled

💬 **INTERVIEW** : controlled vs uncontrolled inputs

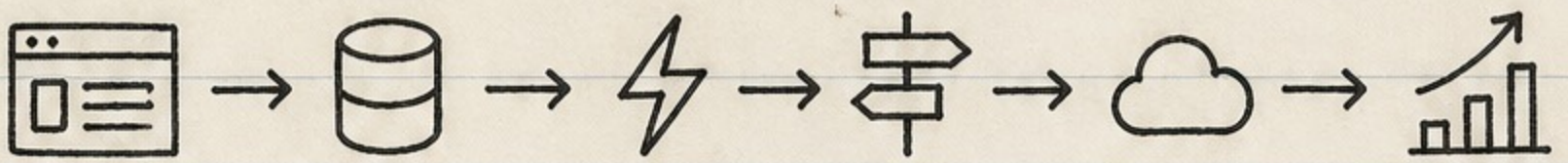
⊗ **MISTAKE** : forgetting preventDefault; reading DOM instead of state

REACT

BEGINNER → ADVANCED

— COMPLETE PREMIUM NOTES —

Build UIs as a function of state — components, hooks, and real app patterns.



Learn once. Think in React.

JSX • Hooks • Context • Router • Fetching • Performance

WHAT'S INSIDE:

- JSX
- Props
- useState
- Events
- Lists & keys
- useEffect
- Custom hooks
- Context
- Refs
- Memo
- Router
- Data fetching
- Patterns
- Interview recap

BY THE NUMBERS:

- 40+ concepts
- Hooks deep dive
- Interview ready
- Beginner to advanced
- Real examples
- Common mistakes

TOPICS COVERED

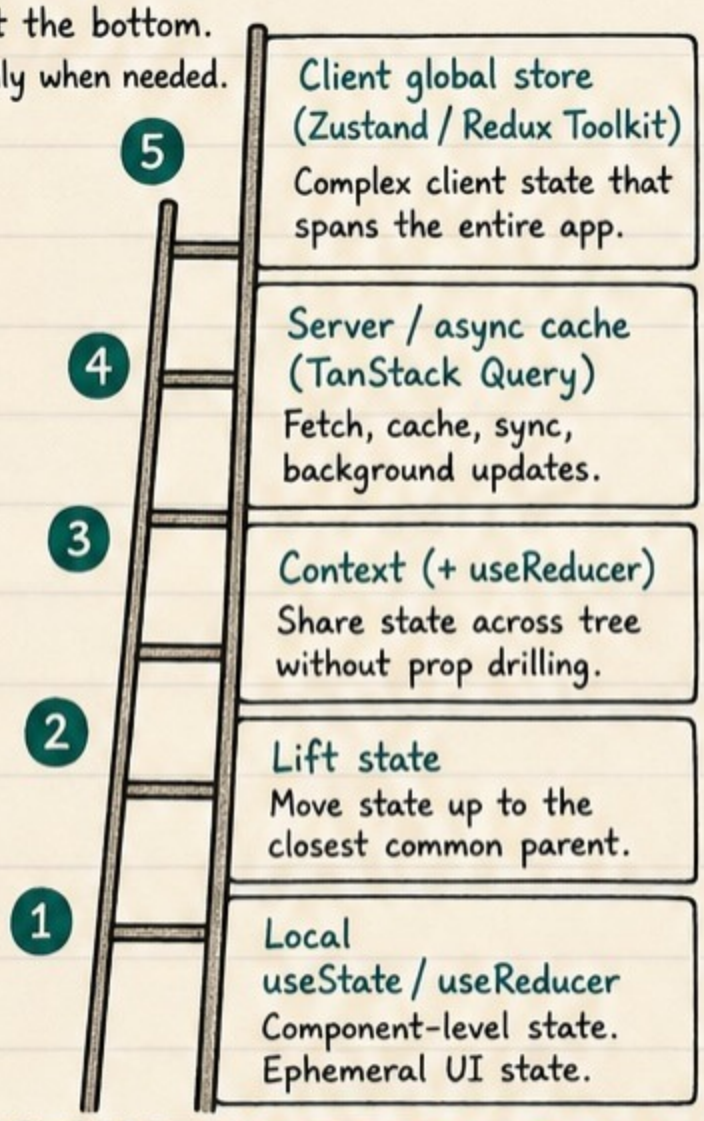
- ✓ Components
- ✓ State & effects
- ✓ Context API
- ✓ React Router
- ✓ Rendering rules
- ✓ Performance
- ✓ Patterns
- ✓ Roadmap

16 — STATE MANAGEMENT CHOICES

Intro: Pick the smallest tool that fits.
Not every app needs Redux.

THE DECISION LADDER

Start at the bottom.
Climb only when needed.



More power, more cost.

Default to the lowest rung that solves the problem.
You can always climb later.

QUICK COMPARISON

Local	simple UI state
Context	theme / auth
Query	server data
Zustand / Redux	complex client cross-page state

useReducer — WHEN?

Use when:

- state has many related transitions
- next state depends on previous state
- logic is getting messy with many setState calls (e.g., > 8–10)
- you want a reducer that's easy to test & reason about

Rule of thumb: If you need 10+ setStates, reach for useReducer.

EXAMPLE — Cart



1) Start Local

- items in component
- add/remove/update
- totals computed locally

→

2) Shared in App?

Header (count) + Product page + Checkout page all need access.

→ lift to Context or small store.

→

3) Server Involved?

Persisted cart, prices, inventory, user carts across devices?

→ use Query for server data.

Start local.
Promote if multiple parts need it.
Use Query for server data.

INTERVIEW ANGLE

- Redux vs Context?
  - Redux = global store with more structure, devtools, middlewares.
  - Context = great for less frequent updates (theme, auth, i18n).
- Where does server state live?
  - On the server + client cache

COMMON MISTAKES

- Using Redux for every input (e.g., text field). Overkill.
- Duplicating server data in Redux without need.
  → leads to stale, complex, hard-to-sync state.
- Ignoring caching and

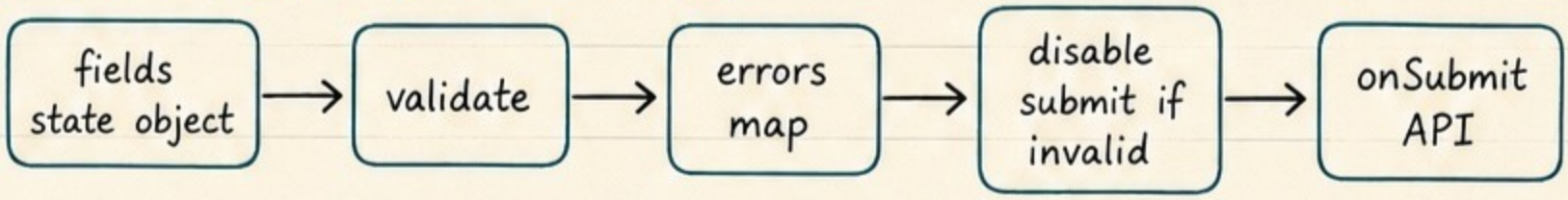
DEEP NOTES

- UI state = ephemeral, view concerns, interactions, toggles.
- Server state = data from backend, needs caching, syncing, invalidation.
- Keep boundaries clean. Derive UI state from server state—don't duplicate.
- Prefer co-located state. Avoid global

14 — FORMS ADVANCED

Real forms need validation, UX feedback, and a clear submit path.

CONTROLLED FORM ARCHITECTURE



VALIDATION LAYERS

1. HTML required/type
   (input required, type="email")
2. Sync rules
   (email format, min length, etc.)
3. Async
   (username taken)

UX CHECKLIST

- ☑ Labels for every field
- ☑ Errors near fields
- ☑ Touched vs submit-time errors
- ☑ Loading state on submit
- ☑ Success toast / message

☆ **LIBS NOTE:** React Hook Form / Formik reduce re-renders and boilerplate. Great for large forms. (Optional, advanced)

EXAMPLE Signup { name, email, password }

```
const [form, setForm] = useState({
  name: '',
  email: '',
  password: ''
});
const [errors, setErrors] = useState({};
```

Name
Jane Doe
Email
jane@example
errors.email → "Please enter a valid email."
Password
•••••••
Sign Up

Example errors object:

```
{
  email: "Please enter a valid email.",
  password: "Min 8 characters"
}
```

INTERVIEW FOCUS

- Controlled vs Uncontrolled tradeoffs
- Why single state object vs many useStates?

Single object → easier to update, validate together, pass around, and reset.

Many useStates → more re-renders, harder to manage at scale.

COMMON MISTAKES

- Validate only on server (bad UX)
- Clearing errors poorly (errors vanish randomly or too early)
- Storing sensitive data (like password) in URL / query params

ADVANTAGES

- Full control over values
- Real-time validation
- Better UX feedback
- Easier to show/hide fields

LIMITATIONS

- Re-renders on each keystroke
- Can be verbose without helpers (optimize with

07 — useEffect & SIDE EFFECTS

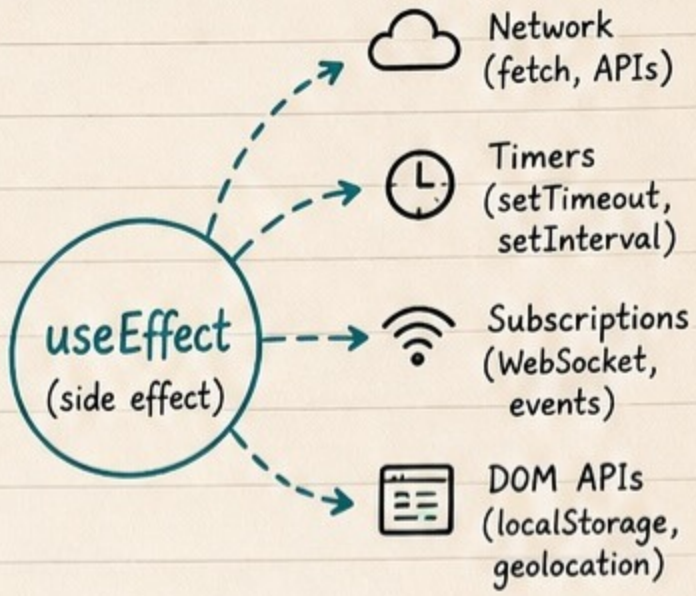
★ Effects sync React with external systems: network, timers, subscriptions, DOM APIs.

DEFINITION

useEffect(fn, deps)
runs after paint;
cleanup optional ☆

WHY?

- keep render pure
- put side effects outside render body



HOW? WORKFLOW



DEPENDENCY ARRAYS

`[]`	mount once (+ cleanup on unmount) ☆
`[id]`	re-run when id changes ☆
no array	every render (usually avoid) ☆

CODE

```
useEffect(() => {
  const id = setInterval(() => {
    // do something
  }, 1000);

  return () => clearInterval(id);
}, []);
```

→ mount once + cleanup on unmount

REAL EXAMPLE

Fetch user by id.
Abort / cleanup on id change.

```
useEffect(() => {
  const controller = new AbortController();
  fetch(`/api/users/${id}`, {
    signal: controller.signal,
  })
    .then(res => res.json())
    .then(setUser)
    .catch(err => {
      if (err.name !== 'AbortError')
        console.error(err);
    });

  return () => controller.abort();
}, [id]); // re-run when id changes
```

cleanup aborts previous request

INTERVIEW NOTES

- useEffect vs useLayoutEffect
  - useEffect runs after paint (async). Good for non-blocking side effects.
  - useLayoutEffect runs before paint (sync) Use for layout reads/writes (measurements), to avoid flicker.

- Stale closures
  Effects capture values from the render.

COMMON MISTAKES

- ✕ Fetching without cleanup → memory leaks, race conditions.
- ✕ Putting everything in effects → overuse. If it can be done during render, do it there.
- ✕ Missing deps / disabling eslint blindly → stale bugs that are hard to trace.

17 — PERFORMANCE PLAYBOOK



Fast UI feels instant.
Fix real bottlenecks, not imaginary ones.

☑ CHECKLIST

1. Reproduce slow interaction
2. React Profiler / what re-renders
3. Reduce state surface
4. Memo hot children
5. Virtualize long lists
6. Code-split routes (lazy + Suspense)

COMMON FIXES

- ✓ state too high → move down
- ✓ context value new object every render → memoize value `{ a: 1, b: 2 }`
- ✓ giant lists → windwoing
- ✓ huge bundles → React.lazy

```
</> CODE
const Page = lazy(() => import('./Page'))
```

? INTERVIEW

- reconciliation: how React turns changes in state/props into minimal DOM updates.
- keys: help React identify items across renders; stable keys = correct + fast updates.
- why virtual DOM helps: diff in memory → batch minimal real DOM updates.
- concurrent features (high-level): React 18+ lets work be interruptible, prioritized, and batched for smoother UI.

18+

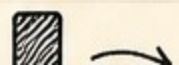
✗ MISTAKE

- micro-memo everywhere (useMemo/useCallback abuse) — complexity & stale bugs.
- ignoring network as real bottleneck (slow APIs > render cost)



★ ADVANTAGES OF LAZY LOADING

- smaller initial JS → faster first paint
- load code only when needed



⚠ LIMITS

- loading spinners / skeletons are needed
- too many splits → waterfall requests

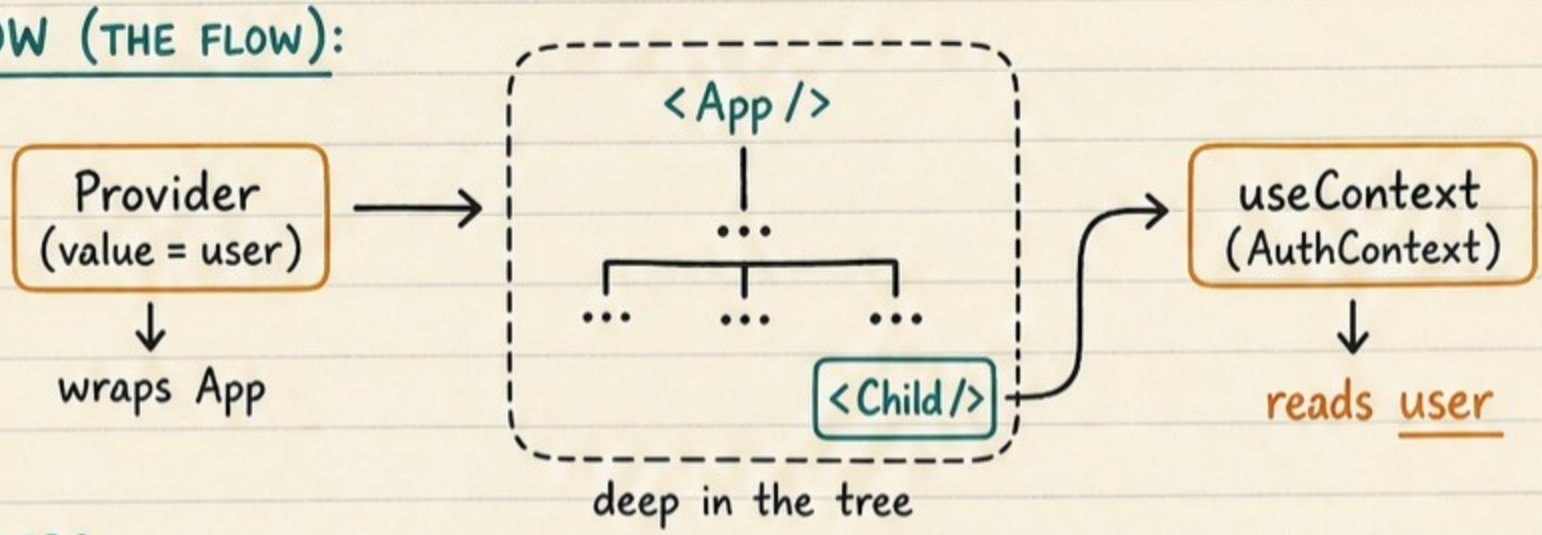
09 — CONTEXT API

✳ Context passes data deep in the tree without prop drilling.

DEFINITION: createContext → Provider value → useContext (Consumer)

WHY: theme, auth user, locale needed by many nested components

HOW (THE FLOW):



STEPS:

1. `const AuthContext = createContext(null)`
2. `<AuthContext.Provider value={user}>`
   `{/* ...children */}`
   `</AuthContext.Provider>`
3. `const user = useContext(AuthContext)`

SPLIT CONTEXT TIP:
Keep context focused.
e.g. ThemeContext
AuthContext
LocaleContext
(separate providers)

WHEN TO USE:
rarely changing global-ish UI data

WHEN NOT TO USE:
high-frequency updates (causes wide re-renders)
→ prefer local state or store

INTERVIEW NOTES:
- Know Context vs Redux/Zustand.
- Understand performance caveats and re-render behavior.

COMMON MISTAKES:
- Putting all app state in one giant context.
- Creating new object/array value every render without memo → causes unnecessary re-renders.

EXAMPLE BAD:
`<Provider value={{ user, theme }}>`
↓